# 实验6 系统调用

- 实验目的

学习如何产生一个系统调用，以及怎样通过往内核中增加一个新函数，从而在内核空间中实现对用户空间的读/写。学习重建内核。

# 实验内容与要求

（1）设计并实现一个新的系统调用pedagogictime()，该函数通过使用一个引用参数的调用返回当前的系统时间。

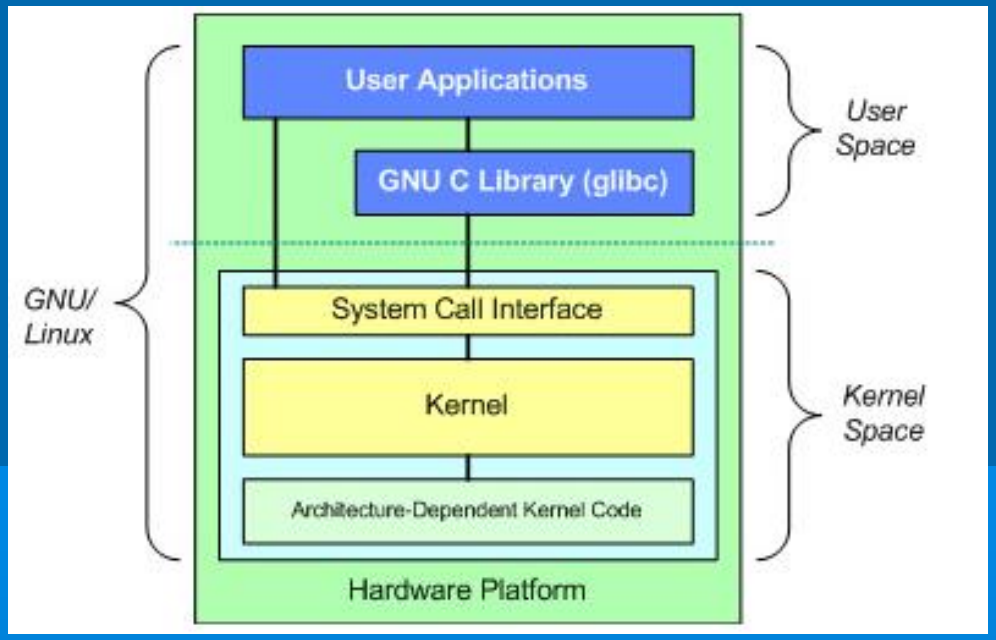
（2）编写一个用户空间程序来测试pedagogictime()。

# 超级（root）用户

超级用户是一种用于进行系统管理的特殊用户，相比其他普通用户来说，它拥有最高权限，能够进行全系统的配置、维护等工作，做很多普通用户没有权限做的事情；而普通用户的权限一般是超级用户的子集，只具备其部分权限。Linux下默认按用户名和密码登陆的，都是普通用户，显示符号位$；超级用户显示符号位#。

普通用户进入超级用户：
oslinux@oslinux-virtual-machine:~$ sudo –s
回车，再输入登陆密码。
超级用户切换到普通用户：
root@oslinux-virtual-machine:~# su oslinux(用户名)
**或者直接退出：**
root@oslinux-virtual-machine:~# exit

# Linux操作系统的基本体系结构




上面是用户（或应用程序）空间。这是用户应用程序执行的地方。

用户空间之下是内核空间，Linux 内核正是位于这里。

# 系统调用

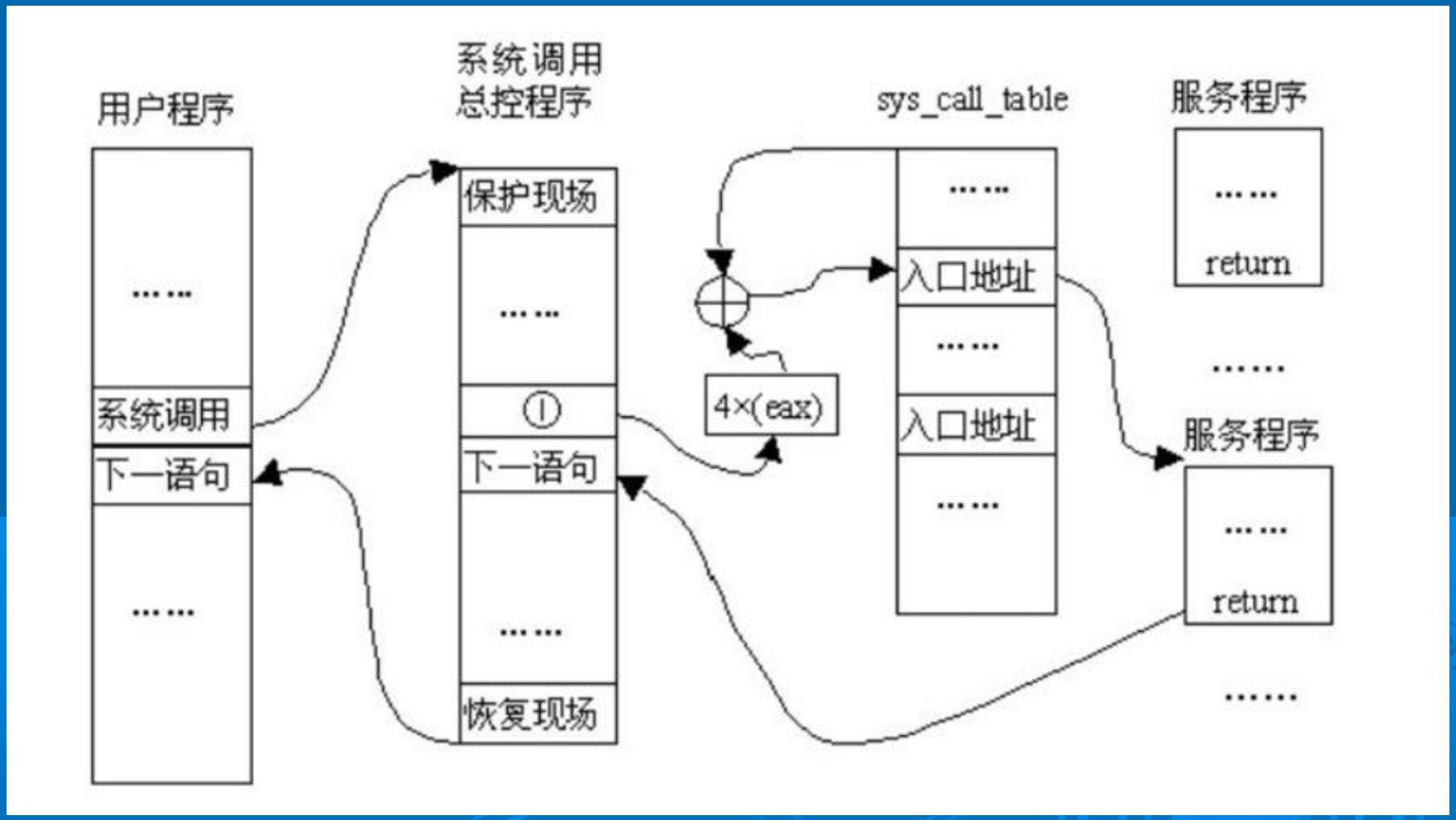
运行在内核态的进程可以毫无限制的访问各种资源，而在用户态下的用户进程的各种操作都有着限制，对资源的访问都必须有操作系统控制。也就是说操作系统是使用这些资源的唯一入口，而这个入口就是操作系统提供的系统调用（System Call）。

简单来说，系统调用把应用程序的请求传给内核，调用相应的的内核函数完成所需的处理，将处理结果返回给应用程序。

**举例：**

| | |
|---|---|
| fork | 创建一个新进程 |
| getpgid | 获取指定进程组标识号 |
| kill | 向进程或进程组发信号 |

# 系统调用过程


用户程序
……
系统调用
下一语句
……
系统调用
总控程序
保护现场
……
①
下一语句
……
恢复现场
4×(eax)
sys_call_table
……
入口地址
……
入口地址
……
服务程序
……
return
……
服务程序
……
return
……

# 系统调用号

每个系统调用都有相应的系统调用号作为唯一的标识，内核维护一张系统调用表，表中的元素是系统调用函数的起始地址，而系统调用号就是系统调用在调用表的偏移量。在进行系统调用是只要指定对应的系统调用号，就可以明确的要调用哪个系统调用，这就完成了系统调用的函数名称的转换。

[cpp] view plain copy print ?

```cpp
#ifndef _ASM_X86_UNISTD_32_H
#define _ASM_X86_UNISTD_32_H

/*
 * This file contains the system call numbers.
 */

#define __NR_restart_syscall      0
#define __NR_exit                 1
#define __NR_fork                 2
#define __NR_read                 3
#define __NR_write                4
#define __NR_open                 5
```

# 系统调用表

为了将系统调用号和相应的服务例程结合起来，内核利用了系统调用表，这个表放在sys_call_table数组中，它是一个函数指针，每一个函数指针都指向其系统调用的封装例程。以本次实验要用的内核为例，它定义在 /usr/src/linux-2.6.32.60/arch/x86/kernel/syscall_table_32.S中。

syscall_table_32.S

```
ENTRY(sys_call_table)
        .long sys_restart_syscall       /* 0 - old "setup()" system call, used for restarting */
        .long sys_exit
        .long ptregs_fork
        .long sys_read
        .long sys_write
        .long sys_open          /* 5 */
        .long sys_close
        .long sys_waitpid
```

# 系统调用参数传递

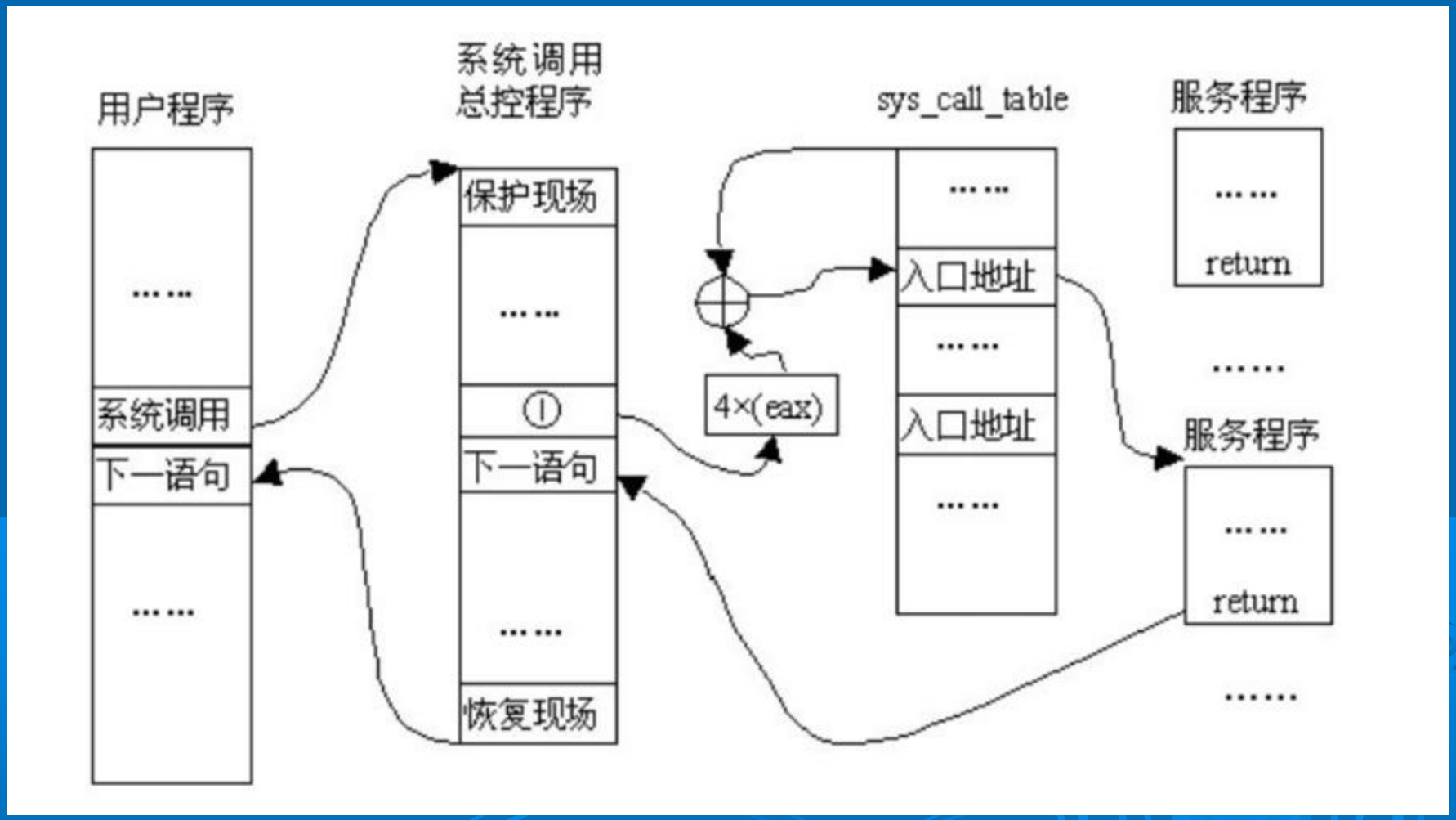
Linux中是通过寄存器%eax传递系统调用号，所以具体调用fork的过程是：将2存入%eax中，然后进行系统调用，伪代码：

[plain] view plain copy print ?

```
01. mov     eax, 2
02. int     0x80
```

对于参数传递，Linux是通过寄存器完成的。Linux最多允许向系统调用传递6个参数，分别依次由%ebx，%ecx，%edx，%esi，%edi和%ebp这个6个寄存器完成。

# 系统调用过程


用户程序
……
系统调用
下一语句
……
系统调用
总控程序
保护现场
……
①
下一语句
……
恢复现场
4×(eax)
sys_call_table
……
入口地址
……
入口地址
……
服务程序
……
return
……
服务程序
……
return
……

# 实验步骤

## 解压内核：

内核文件在文件目录Documents下，将压缩包解压到/usr/src/目录下。

（1） oslinux@oslinux-virtual-machine:~$ cd Documents

//进入Documents目录下

（2） oslinux@oslinux-virtual-machine:~/Documents$ sudo cp linux-2.6.32.60.tar. bz2 /usr/src/

//将压缩包拷贝到/usr/src/目录下

（3） oslinux@oslinux-virtual-machine :~/Documents$ cd /usr/src/ //进入 /usr/src/目录下

（4） oslinux@oslinux-virtual-machine:/usr/src$ sudo tar -jxvf linux-2.6.32.60.tar.bz2 //解压压缩包

# 实验步骤

## 添加系统调用：

### （1）编写系统调用服务例程

编写加到内核中的源程序，即将要加到一个内核文件中去的一个函数，该函数的名称应该是新的系统调用名称前面加上sys_标志。

注意要在/usr/src/linux-2.6.32.60/kernel/sys.c文件中添加源代码。

# 实验步骤

//用编辑器打开sys.c文件添加系统调用pedagogictime的代码

oslinux@oslinux-virtual-machine:~$ sudo gedit /usr/src/linux-2.6.32.60/kernel/sys.c

//添加到sys.c的最后一行,代码如下:

```
asmlinkage long sys_pedagogictime(struct timeval *tv)
{
    if(likely(tv)) {
        struct timeval ktv;
        do_gettimeofday(&ktv);
        if(copy_to_user(tv,&ktv, sizeof(ktv)))
            return -EFAULT;
    }
    return 0;
}
```

# 实验步骤

## （2）添加系统调用号

为了从已有的内核程序中增加到新的函数的连接，需要编辑两个文件：

在/usr/src/linux-2.6.32.60/arch/x86/include/asm/unistd_32.h中增加新的系统调用号

同时还要将宏变量NR_syscalls的值加1；

oslinux@oslinux-virtual-machine:~$ sudo gedit /usr/src/linux-2.6.32.60/arch/x86/include/asm/unistd_32.h

```
#define __NR_perf_event_open    336
#define __NR_pedagogictime      337

#ifdef __KERNEL__

#define NR_syscalls 338
```

# 实验步骤

**（3）修改系统调用表**

在/usr/src/linux-2.6.32.60/arch/x86/kernel/syscall_table_32.S中增加新的内核函数的指针。

oslinux@oslinux-virtual-machine:~$ sudo gedit /usr/src/linux-2.6.32.60/arch/x86/kernel/syscall_table_32.S

```
        .long sys_rt_tgsigqueueinfo     /* 335 */
        .long sys_perf_event_open
        .long sys_pedagogictime /*337*/
```

# 实验步骤

**（4）重新编译Linux内核**

在当前工作目录（/usr/src/linux-2.6.32.60）进入超级用户，才可以重建内核。

oslinux@oslinux-virtual-machine:~$ cd /usr/src/linux-2.6.32.60

oslinux@oslinux-virtual-machine:/usr/src/linux-2.6.32.60$ sudo –s

现在开始编译内核。

# 实验步骤

编译内核的基本过程为：

root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make mrproper //清除依赖

root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
cp /boot/config-3.2.0-29-generic-pae .config

root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make oldconfig //更改配置（一直回车)

root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make clean //清除中间文件

# 实验步骤

root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make bzImage //编译内核

root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make modules //编译内核模块

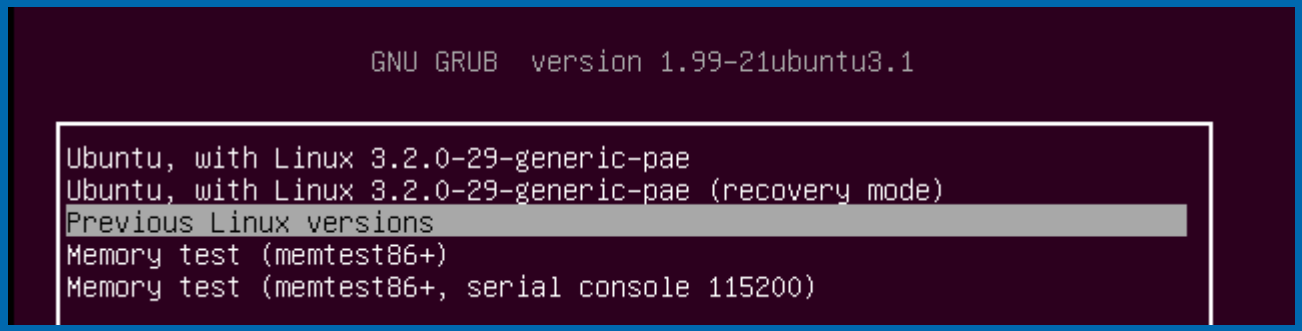
root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make modules_install //安装内核模块

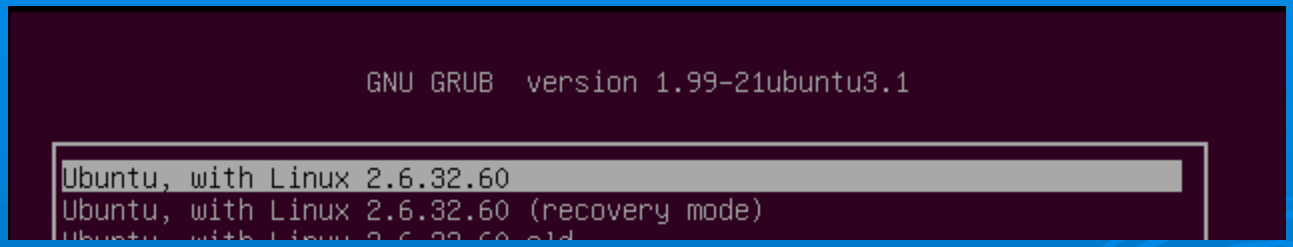
root@oslinux-virtual-machine:/usr/src/linux-2.6.32.60#
make install //安装内核

# 实验步骤

## （5）启用新内核

重新启动，开机时长按shift键直到进入启动加载页面，选择新编译的内核版本，按回车键确认选择并进入 系统 。



# 实验步骤

查看内核版本，是否为新内核版本

oslinux@oslinux-virtual-machine:~$ uname –a

```
oslinux@oslinux-virtual-machine:~$ uname -a
Linux oslinux-virtual-machine 2.6.32.60 #1 SMP Sat Oct 22 23:43:17 CST 2016 i686
 athlon i386 GNU/Linux
oslinux@oslinux-virtual-machine:~$
```

# 实验步骤

## （6）测试

编写测试程序，测试新的系统调用是否添加成功。

```
#include <sys/syscall.h>
#include <linux/unistd.h>
#include <stdio.h>
#include <sys/time.h>
#include <time.h>
main()
{
    struct timeval tv;
    long tmp;
    tmp=syscall(337,&tv);
    if(tmp!=-1)
    {
        printf("First, user get tv_sec:%d\n",(int)tv.tv_sec);
    }
}
```